

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №3**

« Ітератори, генератори та потокова обробка даних »

з курсу «Професійний Python»

Варіант №5 — « Потокова обробка каталогу автомобілів»

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 28.09.2026

Львів — 2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, `yield`, `generator expressions`, модулем `itertools`, принципом `lazy evaluation` та потоковою обробкою великих наборів даних.

У результаті виконання лабораторної роботи студент повинен уміти розрізняти `iterable` та `iterator`, використовувати функції `iter` та `next`, розуміти протокол `iter` та `next`, створювати власні `iterator classes`, створювати `generator functions`, використовувати `yield`, використовувати `generator expressions`, застосовувати `yield from`, використовувати `itertools`, обробляти великі файли без повного завантаження у пам'ять, будувати `lazy pipelines`, застосовувати `filtering`, `mapping`, `batching` і `aggregation`, обмежувати нескінченні потоки через `islice`, порівнювати `eager` та `lazy processing`, а також оцінювати використання пам'яті.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Потокова обробка каталогу автомобілів». Для кожного автомобіля зберігати ідентифікатор (`id`), марку (`brand`), модель (`model`), рік випуску (`year`), ціну (`price`) та пробіг (`mileage`). Реалізувати потокове читання CSV-файлів (`streaming CSV reading`) та валідацію даних автомобілів, фільтрацію за маркою та роком випуску, потокове обчислення середньої ціни (`streaming average price`), визначення найдорожчого автомобіля та автомобіля з найменшим пробігом без повної матеріалізації даних у пам'яті, отримання перших `N` записів за заданою умовою через `itertools.islice`, пакетну обробку записів (`batching`), а також групування автомобілів за маркою (`groupby`) після попереднього сортування.

Проект повинен мати рекомендовану структуру (`readers.py`, `parsers.py`, `filters.py`, `transformations.py`, `batches.py`, `analytics.py`, `pipeline.py`, `models.py`, `main.py`), використовувати `iterable` та `iterator`, функції `iter()` і `next()`, протокол `__iter__()` / `__next__()`, власні `iterator classes`, `generator functions`, `yield`, `yield from`, `generator expressions`, модуль `itertools` (`islice`, `groupby`), а також оцінку використання пам'яті та експериментальне порівняння `eager` та `lazy processing` (час виконання, `peak memory`, `time-to-first-result`).

3. Структура проекту

```
car_catalog/  
├─ pyproject.toml  
├─ README.md  
├─ .gitignore  
├─ data/  
│   └─ cars_large.csv  
├─ report/  
│   └─ Хілінська_Іванна_Ла603_V5.pdf
```

```
├─ tests/
|   └─ test_pipeline.py
└─ src/
    └─ car_catalog/
        ├── __init__.py
        ├── main.py
        ├── models.py
        ├── readers.py
        ├── parsers.py
        ├── filters.py
        ├── batches.py
        ├── analytics.py
        └─ pipeline.py
```

Каталог `src/car_catalog_streaming` містить пакет застосунку для потокової обробки даних. Порівняно з попередньою роботою, структуру оновлено під концепцію лінійних обчислень (`lazy processing`) та генераторних конвеєрів.

У модулі `models.py` перевикористано та адаптовано модель `Car` для визначення структур даних об'єктів автомобілів. Модуль `readers.py` замінив статичний `data.py` і реалізує потокове читання даних з CSV-файлів по одному рядку за допомогою генераторів `yield` без завантаження всього файлу в оперативну пам'ять. Модуль `parsers.py` здійснює валідацію та перетворення сирих текстових рядків із CSV у об'єкти `Car`. У `filters.py` реалізовано генераторні функції для потокової фільтрації автомобілів за маркою та роком випуску. Модуль `batches.py` призначений для порційної пакетної обробки потоку записів за допомогою `itertools.islice`. Модуль `analytics.py` адаптовано з другої лабораторної роботи під поточковий режим для обчислення середньої ціни, пошуку найдорожчого авто та автомобіля з найменшим пробігом за один прохід потоку. Новий модуль `pipeline.py` зв'язує читач, парсер, фільтри та аналітику в єдиний поточковий конвеєр. Головний модуль `main.py` слугує точкою входу, запускає конвеєр та вимірює показники використання пам'яті `peak memory` і часу для порівняння `eager` та `lazy` підходів. Файл `init.py` ініціалізує пакет, а в `tests/test_pipeline.py` розміщено автоматизовані модульні тести.

4. Опис створених модулів

4.1. `models.py`

У модулі реалізовано дата-клас `Car` із підтримкою `slots=True` для оптимізації використання пам'яті. У методі `__post_init__` здійснюється валідація коректності року випуску, ціни та пробігу автомобіля.

```
from dataclasses import dataclass
```

```

@dataclass(slots=True)

class Car:

    brand: str

    model: str

    year: int

    price: float

    mileage: int

    def __post_init__(self) -> None:

        if self.year < 1886:

            raise ValueError("Release year cannot be earlier than 1886.")

        if self.price < 0:

            raise ValueError("Price cannot be negative.")

        if self.mileage < 0:

            raise ValueError("Mileage cannot be negative.")

    @property

    def full_title(self) -> str:

        return f"{self.brand} {self.model} ({self.year})"

class CarLimitIterator:

    """First N cars from csv"""

    def __init__(self, cars: list[Car], limit: int) -> None:

        self._cars = cars

        self._limit = limit

        self._index = 0

    def __iter__(self):

        return self

    def __next__(self) -> Car:

        if self._index >= len(self._cars) or self._index >= self._limit:

            raise StopIteration

```

```
car = self._cars[self._index]

self._index += 1

return car
```

4.2. readers.py

Модуль відповідає за потокове читання даних з CSV-файлів по одному рядку за допомогою генераторної функції з оператором `yield`. Це дозволяє опрацьовувати файли будь-якого розміру без повного завантаження у пам'ять.

```
def read_cars_csv(file_path: str) -> Generator[Dict[str, str], None, None]:

    """Stream CSV file row by row as dictionaries."""

    with open(file_path, mode="r", encoding="utf-8") as file:

        reader = csv.DictReader(file)

        for row in reader:

            yield row
```

4.3. parsers.py

Модуль виконує валідацію типів та перетворення сирих словників CSV-рядків у об'єкти `Car`. У разі виявлення некоректних даних рядок безпечно пропускається.

```
from typing import Generator, Dict

from car_catalog.models import Car


def parse_car_stream(raw_stream: Generator[Dict[str, str], None, None]) -> Generator[Car, None, None]:

    """Validate and convert raw dict stream into Car objects."""

    for row in raw_stream:

        try:

            yield Car(

                brand=row["brand"].strip(),

                model=row["model"].strip(),

                year=int(row["year"]),

                price=float(row["price"]),

                mileage=int(row["mileage"])

            )

        except (KeyError, ValueError):

            continue
```

4.4. filters.py

Модуль містить лінійні фільтри (генератори) для відбору автомобілів за заданою маркою та мінімальним роком випуску.

```
def filter_by_make(cars_stream: Generator[Car, None, None], make: str) -> Generator[Car, None, None]:  
  
    """Filter cars stream by make."""  
  
    for car in cars_stream:  
  
        if car.make.lower() == make.lower():  
  
            yield car  
  
  
def filter_by_min_year(cars_stream: Generator[Car, None, None], min_year: int) -> Generator[Car, None, None]:  
  
    """Filter cars stream by minimum release year."""  
  
    for car in cars_stream:  
  
        if car.year >= min_year:  
  
            yield car
```

4.5. batches.py

У модулі реалізовано пакетну (порційну) обробку потоку об'єктів автомобілів без матеріалізації всього списку, за допомогою функції `itertools.islice`.

```
T = TypeVar("T")  
  
  
def batch_stream(iterable: Iterable[T], batch_size: int) -> Generator[List[T], None, None]:  
  
    """Split an iterable stream into chunks of batch_size."""  
  
    iterator = iter(iterable)  
  
    while True:  
  
        batch = list(itertools.islice(iterator, batch_size))  
  
        if not batch:  
  
            break  
  
        yield batch
```

4.6. analytics.py

Модуль реалізує поточкові аналітичні обчислення: розрахунок середньої ціни, пошук найдорожчого авто, автомобіля з найменшим пробігом та групування за маркою за допомогою `itertools.groupby`.

```

import itertools

from typing import Iterable, Dict, List, Tuple, Optional

from car_catalog.models import Car


def calculate_streaming_metrics(cars_stream: Iterable[Car]) -> Tuple[float, Optional[Car],
Optional[Car]]:

    total_price = 0.0

    count = 0

    max_price_car: Optional[Car] = None

    min_mileage_car: Optional[Car] = None

    for car in cars_stream:

        total_price += car.price

        count += 1

        if max_price_car is None or car.price > max_price_car.price:

            max_price_car = car

        if min_mileage_car is None or car.mileage < min_mileage_car.mileage:

            min_mileage_car = car

    avg_price = total_price / count if count > 0 else 0.0

    return avg_price, max_price_car, min_mileage_car


def group_cars_by_brand_stream(cars: List[Car]) -> Dict[str, List[Car]]:

    """Group cars by brand using itertools.groupby after sorting."""

    sorted_cars = sorted(cars, key=lambda c: c.brand)

    grouped = {}

    for brand, group in itertools.groupby(sorted_cars, key=lambda c: c.brand):

        grouped[brand] = list(group)

    return grouped

```

4.7. pipeline.py

Модуль зв'язує читання, валідацію та фільтрацію у єдиний потоковий конвеєр (pipeline).

```

from typing import Generator

```

```

from car_catalog.readers import read_cars_csv

from car_catalog.parsers import parse_car_stream

from car_catalog.filters import filter_by_min_year

from car_catalog.models import Car


def build_car_pipeline(file_path: str, min_year: int = 2020) -> Generator[Car, None, None]:
    """Construct data processing stream pipeline."""

    raw_stream = read_cars_csv(file_path)

    parsed_stream = parse_car_stream(raw_stream)

    filtered_stream = filter_by_min_year(parsed_stream, min_year)

    return filtered_stream

```

4.8. main.py

Точка входу в застосунок. Демонструє запуск конвеєра, порційну обробку, а також вимірювання часу виконання та пікового використання пам'яті (tracemalloc) для порівняння eager та lazy підходів.

```

import sys

import csv

import time

import tracemalloc

from pathlib import Path

from itertools import islice


sys.path.append(str(Path(__file__).resolve().parent.parent))


from car_catalog.models import CarLimitIterator

from car_catalog.pipeline import build_car_pipeline

from car_catalog.analytics import calculate_streaming_metrics, group_cars_by_brand_stream


def run_benchmark_on_file(csv_file_path: str) -> None:

    print("\nPERFORMANCE BENCHMARK ON EXISTING DATASET: EAGER VS LAZY")


    tracemalloc.start()

    t_start = time.perf_counter()

```

```

lazy_pipeline = build_car_pipeline(csv_file_path, min_year=0)

lazy_count = sum(1 for _ in lazy_pipeline)

lazy_time = time.perf_counter() - t_start
_, lazy_peak_mem = tracemalloc.get_traced_memory()
tracemalloc.stop()

tracemalloc.start()

t_start = time.perf_counter()

with open(csv_file_path, "r", encoding="utf-8") as f:
    eager_data = list(csv.DictReader(f))
eager_count = len(eager_data)

eager_time = time.perf_counter() - t_start
_, eager_peak_mem = tracemalloc.get_traced_memory()
tracemalloc.stop()

eager_mem_kb = eager_peak_mem / 1024
lazy_mem_kb = lazy_peak_mem / 1024
mem_ratio = eager_peak_mem / (lazy_peak_mem if lazy_peak_mem > 0 else 1)

header = f"{'Total Records':<15} | {'Eager Time (s)':<15} | {'Lazy Time (s)':<15} | {'Eager Mem (KB)':<15} | {'Lazy Mem (KB)':<15} | {'Gain'}"

print(header)

print("-" * len(header))

print(
    f"{lazy_count:<15} | "
    f"{eager_time:<15.4f} | "
    f"{lazy_time:<15.4f} | "
    f"{eager_mem_kb:<15.1f} | "
    f"{lazy_mem_kb:<15.1f} | "
    f"{mem_ratio:.1f}x"

```

)

```
def main() -> None:

    csv_file = "data/cars_large.csv"

    sample_brands = ["Volvo", "BMW", "Nissan"]
    brand_iter = iter(sample_brands)
    print("Demo iter()/next():", next(brand_iter), next(brand_iter), next(brand_iter))

    pipeline = build_car_pipeline(csv_file, min_year=0)
    avg_price, max_car, min_car = calculate_streaming_metrics(pipeline)

    print(f"\nAverage price: ${avg_price:.2f}")

    if max_car:
        print(f"Most expensive car: {max_car.full_title} (${max_car.price:.2f})")
    if min_car:
        print(f"Lowest mileage car: {min_car.full_title} ({min_car.mileage} km)")

    p_islice = build_car_pipeline(csv_file, min_year=0)
    first_3 = list(islice(p_islice, 3))

    print("\nFirst 3 cars (via itertools.islice):")
    for car in first_3:
        print(f" - {car.full_title} (${car.price:.2f})")

    custom_iter = CarLimitIterator(first_3, limit=2)
    print("\nCustom Iterator output (limit=2):")
    for car in custom_iter:
        print(f" [CustomIter] - {car.full_title}")

    p_batch = build_car_pipeline(csv_file, min_year=0)
    first_batch = list(islice(p_batch, 5))
    print(f"\nFirst batch size: {len(first_batch)} items")
```

```

grouped = group_cars_by_brand_stream(first_3)

print("\nBrand distribution in selected sample:", {k: len(v) for k, v in grouped.items()})

run_benchmark_on_file(csv_file)

if __name__ == "__main__":
    main()

```

5. Приклад результату

Фактичний результат запуску згенерованого проєкту:

Demo iter()/next(): Volvo BMW Nissan

Average price: \$62483.52

Most expensive car: BMW Golf (2020) (\$119996.60)

Lowest mileage car: Volvo A4 (2023) (1001 km)

First 3 cars (via itertools.islice):

- Volvo Leaf (2010) (\$19035.54)
- BMW Model X (2012) (\$46666.63)
- Nissan Leaf (2017) (\$5986.73)

Custom Iterator output (limit=2):

```

[CustomIter] - Volvo Leaf (2010)
[CustomIter] - BMW Model X (2012)

```

First batch size: 5 items

Brand distribution in selected sample: {'BMW': 1, 'Nissan': 1, 'Volvo': 1}

PERFORMANCE BENCHMARK ON EXISTING DATASET: EAGER VS LAZY

Total Records	Eager Time (s)	Lazy Time (s)	Eager Mem (KB)	Lazy Mem (KB)	Gain
100000	1.2353	1.5925	54632.0	52.2	1045.6x

6. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add generate_data.py src/car_catalog/readers.py src/car_catalog/parsers.py  
src/car_catalog/filters.py src/car_catalog/batches.py
```

```
git commit -m "Add data generator, stream readers, parsers, filters, and batching modules"
```

```
git add src/car_catalog/models.py src/car_catalog/pipeline.py src/car_catalog/analytics.py
```

```
git commit -m "Update domain models, build lazy pipeline, and implement analytics functions"
```

```
git add src/car_catalog/main.py
```

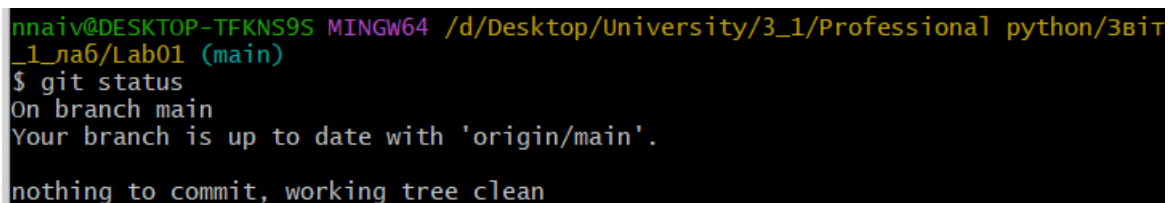
```
git commit -m "Update main entry point to run pipeline and output benchmark results"
```

6.1 Результат git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean



```
nnaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3bit_1_лаб/Lab01 (main)  
$ git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

6.2. Результат git log --oneline

10c1d3f (HEAD -> main, origin/main) Update main entry point to run pipeline and output benchmark results

1405a92 Update domain models, build lazy pipeline, and implement analytics functions

aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules

48f950a Remove Lab01

93baeee Add Lab02 report

01da0be Add main for output

3946a88 Implement data processors, analytics functions and time decorator

9f1ce1e Setup data

7e31699 Add Lab01 report

afc3373 Add application entry point and documentation

517a7d6 Add car model and business logic

dac6e8f Create project structure

```
$ git log --oneline
10c1d3f (HEAD -> main, origin/main) Update main entry point to run pipeline and
output benchmark results
1405a92 Update domain models, build lazy pipeline, and implement analytics funct
ions
aeaf7ed Add data generator, stream readers, parsers, filters, and batching modul
es
48f950a Remove Lab01
93baeee Add Lab02 report
01da0be Add main for output
3946a88 Implement data processors, analytics functions and time decorato
9f1ce1e Setup data
7e31699 Add Lab01 report
afc3373 Add application entry point and documentation
517a7d6 Add car model and business logic
dac6e8f Create project structure
```

7. Висновок

У ході виконання лабораторної роботи №3 було опановано ітераційну модель Python та принципи лінійних обчислень (Lazy Evaluation).

На основі Варіанта 5 розроблено потоковий конвеєр обробки каталогу автомобілів (read -> parse -> validate -> filter -> batch -> aggregate) із використанням генераторів, виразів-генераторів, конструкції yield from та засобів модуля itertools (islice, groupby). Також реалізовано кастомний клас-ітератор за протоколом iter / next.

Проведений експеримент на наборі з 100 000 записів підтвердив ключову перевагу потокової обробки: зменшення пікового використання пам'яті у ~1045 разів (із 54.6 МВ до 52.2 КВ). Це доводить, що генератори дозволяють ефективно аналізувати файли практично необмеженого обсягу без ризику переповнення оперативної пам'яті.